

End-of-Studies Project (PFE)

A Privacy-First, No-Code AI Platform
for Tabular ML, Local Generative AI, and Deep Learning

[Student Name]

Supervised by [Supervisor Name]

May 30, 2026

Abstract

This report documents the design and implementation of a no-code AI platform built across six sprints as an end-of-studies project. The platform lets non-technical users upload data, sketch a pipeline on a visual canvas, and train a model under one fixed constraint: nothing ever leaves the machine. Three workloads share the same canvas. The first is tabular machine learning across fourteen algorithms covering classification, regression, clustering, and forecasting. The second is local Generative AI through a Retrieval-Augmented Generation chat powered by Ollama and `pgvector`. The third is deep-learning image classification on a curated CNN catalogue sized to fit on a 6 GB consumer GPU. The chapters that follow walk through each sprint's design decisions, the failures that taught us something, and the architectural conventions that ended up load-bearing: Celery for every CPU-bound task, MongoDB for anything that resists a schema, and a strict gateway-only header-injection scheme that gave the microservices a zero-trust boundary without operational overhead.

Contents

General Introduction	7
1 General Context of the Project	8
1.1 Project Overview and Context	8
1.1.1 Host context	8
1.1.2 Project description — the no-code AI platform	8
1.2 Requirements Specification	9
1.2.1 Functional Requirements	9
1.2.2 Non-Functional Requirements	10
1.3 Technical Architecture	11
1.3.1 Technology Stack	11
1.3.2 Technical Constraints	11
1.4 Deliverables and Evaluation Criteria	12
1.4.1 Project Deliverables	12
1.4.2 Evaluation Criteria	12
1.5 Interface Mockups	12
1.5.1 Authentication interface	12
1.5.2 Home page interface	13
1.5.3 Admin dashboard interface	13
1.6 Project Duration	13
2 Preliminary Study	14
2.1 Current Situation	14
2.2 Observed Problems	14
2.2.1 Forced data exfiltration	14
2.2.2 Cloud LLM dependence in generative-AI tooling	15
2.2.3 The notebook cliff	15
2.2.4 Hardware ambiguity	15
2.3 Objectives of the New System	15
2.4 État de l’art technique	16
3 Methodology and Working Environment	17
3.1 Comparison of Project Management Methodologies	17
3.2 The Choice of Agile Methodology	18
3.3 Scrum Methodology	18
3.3.1 Scrum Roles	18
3.3.2 Scrum Artefacts	18
3.3.3 Scrum Events	18

3.4	Product Backlog	19
3.5	Project Planning	19
3.6	Technologies	19
3.7	Architecture	20
4	Sprint 1: Authentication, Gateway, and Data Ingestion	22
4.1	Sprint Backlog	23
4.1.1	Use case — Sign in with optional 2FA	24
4.1.2	Use case — Verify email via MailHog	24
4.2	Design	25
4.2.1	Why an opaque-but-decoded gateway	25
4.2.2	Header-injection rules	25
4.2.3	The shared-volume convention	25
4.2.4	Why MongoDB for dataset metadata	25
4.3	Implementation	26
4.3.1	Security posture	27
4.3.2	SQL connectors and Fernet encryption	27
4.4	Unit Tests	27
5	Sprint 2: Visual Pipeline Editor and Tabular Machine Learning	29
5.1	Sprint Backlog	30
5.2	CRISP-DM applied to the tabular ML sprint	30
5.3	Design	31
5.3.1	Graph schema	31
5.3.2	The <code>BaseMLModel</code> abstraction	32
5.3.3	Algorithm selection	32
5.4	Implementation	33
5.4.1	Frontend canvas	33
5.4.2	Preprocessing and feature handling	34
5.4.3	Live training visualisation	34
5.4.4	Evaluation and metric interpretation	34
5.4.5	SHAP and the explainability surface	34
5.4.6	Model export	35
5.4.7	Why <code>joblib</code> and not <code>pickle</code>	35
5.5	Unit Tests	35
6	Sprint 3: Local Generative AI and Retrieval-Augmented Chat	36
6.1	Sprint Backlog	36
6.2	Concepts fondamentaux	37
6.3	Design	37
6.4	Implementation	38
6.4.1	Streamed chat	38
6.4.2	Chunk sizing trade-offs	38
6.4.3	Prompt construction	38
6.5	Unit Tests	39

7	Sprint 4: Deep Learning for Image Classification	40
7.1	Sprint Backlog	40
7.2	Concepts fondamentaux	41
7.3	Design	41
7.3.1	Scope decisions	41
7.3.2	Why CNN architectures, and which three	42
7.3.3	Why a separate microservice	42
7.4	Implementation	43
7.4.1	Preprocessing and augmentation	43
7.4.2	The VRAM guard	43
7.4.3	Tier-aware ceilings	43
7.4.4	Frontend integration	44
7.4.5	Live progress and inference	44
7.4.6	Demo path end-to-end	44
7.5	Unit Tests	45
8	Sprint 5: Multi-tenancy, Project ACL, and Real-Time Collaboration	46
8.1	Sprint Backlog	46
8.2	Concepts fondamentaux	46
8.3	Design	47
8.3.1	The role hierarchy	47
8.4	Implementation	47
8.4.1	Real-time presence	47
8.4.2	Pipeline chat and Google Meet	47
8.4.3	Why the gateway resolves the role	48
8.5	Unit Tests	48
9	Sprint 6: Dashboard, i18n, Accessibility, and Admin Console	49
9.1	Sprint Backlog	49
9.2	Design	50
9.3	Implementation	50
9.3.1	The user dashboard and billing UI	50
9.3.2	Internationalisation	50
9.3.3	High-contrast theme and accessibility	50
9.3.4	Notification bell with persistence	50
9.3.5	Admin operations dashboard	50
9.3.6	User impersonation	51
9.3.7	Undo / redo on the canvas	51
9.3.8	Cross-cutting changes	51
9.4	Unit Tests	51
10	Testing and Deployment	53
10.1	Test Strategy	53
10.1.1	Backend Testing	53
10.1.2	Frontend Testing	54
10.1.3	CI runs	54
10.2	Deployment	54
10.2.1	Volumes	55
10.2.2	Healthchecks and dependency ordering	55

10.2.3	Network and security topology	55
10.2.4	Bugs encountered during deployment	56
	General Conclusion	57

List of Figures

List of Tables

1.1	Functional requirements summary.	9
1.2	Non-functional requirements with concrete commitments.	10
1.3	Technology stack by layer with the role of each component.	11
3.1	Comparison of candidate project management methodologies.	17
4.1	Sprint 1 backlog — foundation services.	23
4.2	Textual description of the “Sign in with optional 2FA” use case.	24
4.3	Textual description of the “Verify email” use case.	24
5.1	Sprint 2 backlog — canvas plus tabular ML stack.	30
5.2	CRISP-DM phases mapped to Sprint 2 deliverables.	31
5.3	Algorithm catalogue for the tabular ML mode.	33
6.1	Sprint 3 backlog — local RAG.	36
7.1	Sprint 4 backlog — deep learning for image classification.	40
7.2	CNN architecture catalogue with operating envelope.	42
8.1	Sprint 5 backlog — multi-tenancy and collaboration.	46
9.1	Sprint 6 backlog — operability and polish.	49
10.1	Backend test coverage by service.	54
10.2	Deployment topology — 14 containers, three named volumes.	55

General Introduction

Machine learning has become routine in almost every industry, but the tooling around it still defaults to one of two extremes. On one end sit the cloud-managed services — DataRobot, Vertex AI, Azure ML Studio — which hide complexity behind polished interfaces and require users to ship their data into a vendor’s infrastructure. On the other end live the open notebooks where everything is possible and nothing is easy: Jupyter, Colab, raw Python. The space between those two poles, where a non-technical user might want to build a model without either learning Python or surrendering their data, is surprisingly thin.

That gap is the question this project answers. The goal was a platform that gives a domain expert — a clinician, an analyst, a small business owner — enough visual scaffolding to build, train, and inspect a model on their own machine. Not a tutorial, not a hosted SaaS, but a real piece of software that runs locally, accepts local data, and never phones home to a vendor API.

Three workloads sit behind the same canvas: tabular ML, RAG-style generative AI, and image classification. Each imposes a different operational profile, and fitting all three into a single coherent product forced choices that were not obvious upfront. The report is organised as follows. Chapter 1 sets the project context. Chapter 2 reviews the existing landscape and the problems the platform addresses. Chapter 3 describes the methodology and the toolchain. Chapters 4 through 9 walk through the six sprints in order: the foundation sprint, the three AI workloads back-to-back (ML, then LLM, then DL), the collaboration layer, and the operability sprint that moved the platform from a working prototype into something we could hand to a stranger. Chapter 10 covers testing and deployment. The General Conclusion steps back to assess what worked, what didn’t, and what we would build with another quarter.

Chapter 1

General Context of the Project

Introduction

This chapter frames the problem the platform is built to address. We describe the host context, the platform itself in plain terms, the functional and non-functional requirements committed to before any code was written, the technical architecture at a high level, and the deliverables the work is evaluated against.

1.1 Project Overview and Context

1.1.1 Host context

The project was developed as an end-of-studies (PFE) project for a National Bachelor's Degree in Information Technologies, specialisation Development of Information Systems. The deliverable is a working software product plus this report. The host setting is academic rather than corporate: the supervisor is the academic advisor, the audience is the jury, and the reference deployment is a personal laptop with a single consumer GPU.

1.1.2 Project description — the no-code AI platform

The platform is a self-hosted web application. A signed-in user can:

- Upload tabular data (CSV, Excel), pull it from a Postgres or MySQL connector, or upload a zip of images organised by class folder.
- Inspect a profile of the dataset — missing values, skew, outliers, target imbalance, correlations — before doing anything else with it.
- Build a pipeline visually on a React Flow canvas, in one of three modes:
 - **Traditional ML** — fourteen supervised and unsupervised algorithms spanning classification, regression, clustering, and time-series forecasting.
 - **Generative AI** — index documents into a local vector store and chat with a RAG pipeline.
 - **Deep Learning** — image classification on a fixed catalogue of three CNN architectures.

- Train, evaluate, download the trained artefact, and — for image models — run live inference on a hand-uploaded test image.
- Collaborate with teammates inside a company workspace, with role-based access control, real-time canvas presence, threaded per-pipeline chat, and one-click Google Meet links.

A super-admin role sits alongside the user-facing application with its own dashboard for user management, audit logs, queue health, hardware monitoring, and a “view-as-user” impersonation flow gated by a hard five-minute TTL.

1.2 Requirements Specification

1.2.1 Functional Requirements

Table 1.1: Functional requirements summary.

Area	Requirement
Tabular ML	Dataset → train → evaluate pipeline backed by fourteen algorithms (classification, regression, clustering, forecasting).
RAG	Document → vector store → streamed chat, end to end on the local box.
Deep Learning	Image dataset → CNN architecture → training loop, fully local.
Authentication	Email/password, optional TOTP, email verification via MailHog, password reset round-trip.
Billing	Stripe-backed free / solo / company tiers, with super-admin overrides on per-user quotas.
Collaboration	Real-time presence and shared editing on the pipeline canvas.
Access control	Per-pipeline ACL with five roles (Owner, Project Manager, Data Scientist, Analyst, Viewer).
Admin operations	User management, audit logs, queue and hardware monitoring, user impersonation with 5-minute TTL.

1.2.2 Non-Functional Requirements

Table 1.2: Non-functional requirements with concrete commitments.

Property	Commitment
Data sovereignty	No outbound calls to AI vendors. Stripe is the only third-party service in the deployment graph, and it is optional — without an API key the platform still works on the free tier.
Modularity	Five independent Python services so a long training run cannot block an authentication request.
Reproducibility	Every dataset version is profiled, every training run is recorded, every model artefact is versioned and downloadable.
Hardware envelope	Single consumer machine: 16 GB RAM, GTX 1660 Super (6 GB VRAM). The platform runs on it, not just compiles on it.
Accessibility	WCAG AA contrast on the default theme; opt-in high-contrast theme; <code>aria-label</code> on every interactive element.
Internationalisation	English and French i18next bundles, with TypeScript bindings generated from the English bundle so a missing key fails the type-check.
Security	JWT decoded in-process at the gateway; <code>X-User-*</code> headers stripped then re-injected; Redis blacklist for refresh tokens; bcrypt-12 password hashing; Fernet-encrypted SQL credentials.
Performance	Live training events streamed over SocketIO at <50 ms cadence; gateway adds <5 ms latency per proxied request.

1.3 Technical Architecture

1.3.1 Technology Stack

Table 1.3: Technology stack by layer with the role of each component.

Layer	Component	Role in this project
Frontend	React 18 + Vite + TypeScript + MUI	SPA shell with strict-mode type checking.
	React Flow	Visual pipeline canvas with custom nodes.
	Plotly + i18next + Zustand	Live charts, EN/FR i18n, state store.
Backend	Flask + Celery + Flask-SocketIO	Five microservices, async queues, real-time channels.
Data	PostgreSQL + pgvector	Auth, ACL, billing, vector store (co-located).
	MongoDB	Datasets, pipelines, task results, anything schemaless.
	Redis	Celery broker / result backend / SocketIO message bus.
	TimescaleDB	Hardware-telemetry time series.
ML	scikit-learn, XGBoost, LightGBM, CatBoost, H2O Prophet	Fourteen tabular algorithms via the BaseMLModel registry. Time-series forecasting with seasonality.
	SHAP	TreeExplainer-fast post-hoc explanations.
LLM	Ollama + Llama 3.2 3B (Q4_K_M) sentence-transformers MiniLM-L6-v2	Local chat completion at ~ 30 tok/s on 6 GB VRAM. 384-dim embeddings, CPU-only.
DL	PyTorch + torchvision (CUDA 12.1)	LeNet, custom ResNet-9, MobileNet-V3-Small.
Ops	Docker Compose	14-container orchestration via single make up .
	GitHub Actions	Parallel lint + test jobs, ~ 14 min full-stack.
	MailHog	Dev email capture for verification + reset flows.

1.3.2 Technical Constraints

The platform operates inside a tight envelope. A single host machine runs every service. A single GPU with 6 GB of VRAM handles every GPU-bound workload. No cloud AI

vendor sits in the request path, and that constraint is hard, not a preference. The full stack must come up with a single `make up` on a clean machine, and continuous integration must complete in under fifteen minutes for a full-stack change. These limits shaped almost every later design choice.

1.4 Deliverables and Evaluation Criteria

1.4.1 Project Deliverables

- A working web application running the three pipeline modes end to end on the demo machine.
- Five Python microservices, packaged as Docker images.
- A React + TypeScript frontend, packaged as a static build.
- A `docker-compose.yml` that brings the entire stack online with one command.
- A CI pipeline (lint + tests, parallel jobs) green on the `main` branch.
- This written report and the supporting diagrams.

1.4.2 Evaluation Criteria

- Functional coverage of the three pipeline modes.
- Soundness of the architecture (separation of concerns, explicit security boundaries, reproducible deployment).
- Code quality (typing, linting, test coverage on the load-bearing paths).
- Quality of the report and the live demo.

1.5 Interface Mockups

The three mockups below frame the visual identity that the rest of the report refers back to. They were drawn before the canvas implementation began and held up almost unchanged through six sprints, with the exception of the admin tab strip, which collapsed from a sidebar into a flat tab bar during the polish sprint.

1.5.1 Authentication interface

The authentication mockup commits to three things: a single combined sign-in/sign-up surface, an optional TOTP entry that only appears once the credentials are validated, and a ‘Forgot your password?’ link that opens the MailHog round-trip described in Chapter 4. The mockup deliberately leaves the visual treatment of the verification banner open, because that landed during Sprint 6 polish.

1.5.2 Home page interface

The signed-in home page sets the layout the rest of the platform inherits: a left sidebar with the seven module entries (Dashboard, Datasets, Pipelines, Models, Collaborator, Billing, Profile), a top bar with the language toggle, contrast toggle, notification bell, tier chip and account menu, and a content area that renders the active page. The mockup commits to the get-started checklist on the dashboard, which was the single change that most improved first-session retention during user testing.

1.5.3 Admin dashboard interface

The admin mockup is deliberately different from the user surface: no sidebar, a four-tab strip (Stats & Logs / User Management / Announcements / Ops Console), and a single red impersonation banner that hangs above every page once a ‘view-as’ session is active. The mockup made it explicit from day one that the admin console is a separate surface, not a privileged view of the user app.

1.6 Project Duration

The project ran across six sprints of roughly two to three weeks each, with a runnable demo and a self-contained git tag at the end of every sprint. The Gantt chart in Chapter 3 captures the planned versus actual timeline.

Conclusion

This chapter framed the problem the rest of the report addresses — how to make machine learning approachable without surrendering data sovereignty — and sketched the platform that emerged in response. The next chapter reviews the existing landscape and explains why the question is harder to answer with off-the-shelf tools than first appears.

Chapter 2

Preliminary Study

Introduction

Before describing how the platform was built, we look at why nothing already on the market quite fits the problem. The no-code ML space is crowded, but the products in it cluster into families that all share the same blind spot.

2.1 Current Situation

Three categories of product dominate the space.

Cloud-native AutoML platforms (DataRobot, H2O Driverless AI, Azure ML Studio, Google Vertex AI) are polished, exhaustive in their algorithm catalogues, and almost invariably SaaS-only. Their privacy story ends at “trust our infrastructure”. For commercial buyers without regulatory exposure that may be acceptable; for the regulated audience this project targets it is not.

Open-source notebooks (Jupyter, Google Colab, DataLab) are powerful for experts. Every workflow eventually bottoms out in raw Python — which is exactly the friction the no-code category was invented to remove. A domain expert who can read a confusion matrix but cannot write `from sklearn.ensemble import RandomForestClassifier` hits a wall on line one of the first cell.

Visual ETL tools (KNIME, Alteryx, RapidMiner) come closest in spirit. Their centre of gravity is data preparation rather than model training, and their LLM panels almost always route requests back through OpenAI. The locality story they tell only holds for the tabular path.

2.2 Observed Problems

2.2.1 Forced data exfiltration

A clinician analysing patient records, or a bank analyst classifying transactions, cannot lawfully upload that data to a vendor bucket. The problem is regulatory, not contractual — GDPR, HIPAA, and country-specific banking rules forbid the upload entirely. Every SaaS AutoML platform listed above assumes the user is fine with that upload. The audience this project targets is not.

2.2.2 Cloud LLM dependence in generative-AI tooling

Visual tools that grew an “AI” panel in the last two years almost always route the request through OpenAI or a similar API. The panel is bolted on, not local. For a buyer who cannot ship the prompt either, the panel is unusable. Worse, those tools rarely document that the user’s prompts are now training data for the vendor.

2.2.3 The notebook cliff

Notebooks solve the locality problem cleanly. They fail on the no-code one. A non-programmer can drive a Jupyter cell only as far as someone else has prewritten the code for them, and at that point the notebook is no longer a notebook but a brittle web form. Closing this gap is the entire reason a visual canvas exists.

2.2.4 Hardware ambiguity

The few self-hostable products that do exist (KubeFlow, MLflow on local Kubernetes) assume the operator has a Kubernetes cluster sitting around. That is not the operating environment of a small team that has a workstation and an ambition. A platform whose deployment story requires cluster administration is not really a self-hostable product; it is a hosted product that pretends.

2.3 Objectives of the New System

The platform sets out to satisfy four objectives at once.

Locality. Every model server, vector store, queue, and database runs inside the same Docker Compose stack on the user’s own machine.

Coverage. Three workloads — tabular ML, RAG-style generative AI, and image classification — share the same canvas. The user does not need a different product for each.

Approachability. A canvas-based UI that a non-programmer can drive end to end, with per-node validation that catches the obvious mistakes (a Train node with no upstream Dataset, an Evaluate node with no completed run) before they reach the worker.

Honest hardware story. The reference deployment is a laptop with 16 GB of RAM and a GTX 1660 Super (6 GB VRAM). The platform must run on it, not just compile on it.

Two technical shifts make the fourth objective realistic today and would not have made it realistic two years ago. Aggressive LLM quantisation has collapsed model footprints. A three-billion-parameter model in FP16 occupies roughly 12 GB of weights; the Q4_K_M four-bit quantisation shrinks that to about 2 GB, while preserving most of the original quality on conversational tasks. The other shift is first-class GPU inference runtimes for consumer hardware. Ollama, llama.cpp, and vLLM all expose efficient batched inference on cards with 4–8 GB of VRAM. A 1660 Super, with 6 GB of VRAM and 30 TFLOPS of FP16 throughput, serves roughly 30 tokens per second on a 3B model — enough latency budget for an interactive RAG chat.

2.4 État de l’art technique

The platform builds on four well-documented technique families. We summarise the relevant prior art here so the sprint chapters can reference it without restating the basics.

Tree-based gradient boosting. XGBoost [2] formalised regularised gradient boosting on decision trees and remains the default for tabular ML competitions. The Tunisian DSI thesis context favours XGBoost over deep tabular models because its training cost fits in minutes on a CPU and its hyperparameter surface is mature.

Post-hoc explainability via Shapley values. The SHAP framework [3] attributes a model’s output to its input features using Shapley values from cooperative game theory. The optimised TreeExplainer variant lifts the cost of SHAP from prohibitive to interactive on tree models — the only reason SHAP is a live feature in this platform rather than a batch job.

Retrieval-Augmented Generation. Lewis et al. [6] introduced the RAG formulation: embed query and corpus, retrieve top-k chunks by similarity, condition a generative model on the retrieved context. This formulation is what makes a local LLM useful on private documents — the model is small (3B), the corpus does the heavy lifting.

Compact CNN architectures. Three architecture families are represented: LeNet [10] as a smoke-test baseline, residual networks [9] as the workhorse, and MobileNetV3 [8] as the transfer-learning option. Each sits at a different parameter/accuracy operating point compatible with the 6 GB VRAM budget.

Conclusion

Cloud platforms solve the wrong half of the problem, notebooks demand too much of the user, and visual ETL tools are not aimed at model training. The platform described in the rest of this report fills that gap deliberately: local by construction, broad enough to cover the three AI workloads that matter today, and modest enough in its hardware appetite to run on a laptop.

Chapter 3

Methodology and Working Environment

Introduction

Six sprints, one team, three AI workloads on a single GPU — the project’s primary risk was scope, not technique. This chapter explains the methodology used to keep that risk in check, the toolchain that anchored the build, and the layout of the services those choices produced.

3.1 Comparison of Project Management Methodologies

Three styles were considered before settling on one. Table 3.1 captures the trade-offs against the constraints of an ML-heavy solo project.

Table 3.1: Comparison of candidate project management methodologies.

Approach	Strengths	Why rejected (or chosen)
Waterfall	Predictable cost, single up-front spec, easy to communicate.	Fails an ML project where the cost of a feature is genuinely unpredictable — Prophet’s CmdStan dependency alone pushed a “trivial” algorithm into half a sprint. Rejected.
Pure Kanban	Continuous flow, no ceremony overhead.	Removes the dated checkpoints needed to demo to a supervisor; produces no natural fallback point if a late feature fails. Rejected.
Scrum-lite	Short fixed-length sprints, runnable demo + git tag at each boundary, written retro.	Enough ceremony to enforce scope discipline without consuming the sprint. Chosen.

Waterfall plans everything upfront, builds in order, and demos at the end. That shape fails an ML project where the cost of a feature is genuinely unpredictable. **Pure Kanban** offers continuous flow with no sprint boundary, which removes exactly the checkpoints needed to demo to a supervisor on a fixed cadence. **Scrum-lite**, the chosen approach, runs short sprints with a runnable demo, a self-contained git tag, and a retrospective note at each boundary — enough ceremony to enforce scope discipline without consuming the sprint.

3.2 The Choice of Agile Methodology

Two reasons pushed the project toward Agile. First, ML capacity planning is unpredictable. A feature that looks easy on paper hides compile-from-source dependencies, missing CUDA wheels, or libraries whose API has shifted between minor versions. Agile’s sprint boundary absorbs that kind of slip. Second, the project needed dated checkpoints with no hand-waving. Without sprint structure the deliverable becomes a single “everything works at once” release, with no story to tell along the way and no fallback when a late feature falls through.

3.3 Scrum Methodology

3.3.1 Scrum Roles

In a small team the roles collapse onto a single person at a time. What does not collapse is the discipline of switching hats. When planning, the operator is the Product Owner — writing user stories and prioritising the backlog. When building, the operator is the Developer. When reviewing the sprint, the operator becomes the Scrum Master — asking what went wrong, what would change next time. Treating those as three roles, even when they share one head, keeps the practice honest.

3.3.2 Scrum Artefacts

- **Product Backlog.** A single Markdown file under `docs/backlog.md`, grouped by sprint.
- **Sprint Backlog.** The slice of the product backlog promised for the current sprint, lifted into the sprint’s section.
- **Increment.** The git tag at the end of the sprint, plus the demo recording.

3.3.3 Scrum Events

- **Sprint Planning.** Half a day at the start of each sprint. Carry-over from the previous sprint plus a fresh slice of the backlog.
- **Daily check-in.** A short written log entry, not a meeting.
- **Sprint Review.** A live demo with the supervisor at sprint end.

- **Sprint Retrospective.** A retro note in the same Markdown backlog, under the sprint heading.

3.4 Product Backlog

Six sprints, each closing with a runnable demo and a self-contained git tag:

- **Sprint 1 — Authentication, Gateway, Data Ingestion.** Foundational service mesh.
- **Sprint 2 — Visual Pipeline Editor and Tabular ML.** Canvas, fourteen algorithms, SHAP, model export.
- **Sprint 3 — Local Generative AI and RAG.** Ollama, pgvector, streamed chat.
- **Sprint 4 — Deep Learning for Image Classification.** PyTorch service, three CNNs, the VRAM guard.
- **Sprint 5 — Multi-tenancy and Real-Time Collaboration.** Project ACL, cursors, threaded chat, Meet links.
- **Sprint 6 — Dashboard, i18n, Accessibility, Admin Console.** Operability and polish.

The three AI workloads sit back-to-back as Sprints 2–4. That order matters: tabular ML grounds the canvas and the algorithm-registry abstraction; RAG extends both into streaming chat; deep learning then reuses the canvas, the registry, and the live-training channel without rebuilding any of them.

3.5 Project Planning

Two sprints slipped by roughly a week each. Sprint 2 ran long because the live training visualisation had a Plotly autoscale bug that took a day to diagnose — progress percentage and metric values lived on incompatible Y-axis ranges. Sprint 4 ran long because the static VRAM estimate needed an empirical recalibration pass after the first real GPU run exceeded the predicted budget by roughly 15% on MobileNet-V3 at 224px batch 32.

3.6 Technologies

Frontend — React 18 + Vite + TypeScript + MUI. React for the ecosystem, Vite because cold starts under `create-react-app` were unworkable, MUI because it ships accessible components that already satisfy WCAG AA without bespoke work.

Backend — Flask + Celery + Flask-SocketIO. Flask was the lowest-common-denominator choice for a microservice. FastAPI would have given typed routes for free, but every template used as a starting point assumed Flask, and rewriting five services for typing was not a fight worth picking. Celery handles every CPU-bound task — profiling,

preprocessing, H2O training, RAG ingestion, image extraction, and PyTorch training all live on their own queue.

Postgres + pgvector for relational and vector data. Co-locating the vector store with the auth schema avoided a sixth backing service. `pgvector`'s IVFFlat index is more than enough for the few hundred thousand chunks a single-tenant local deployment is ever likely to hold.

MongoDB for everything schemaless. Datasets, pipelines, model versions, task results — anything whose shape evolves between sprints lives in Mongo. A Postgres JSONB column was considered for this and rejected: lossless storage, flexible queries, and zero migrations as new statistics get added all favoured Mongo.

Redis for queue + cache + socket message bus. Three logical databases on the same instance — DB 0 for Celery, DB 1 for Celery results, DB 2 for the SocketIO message queue.

Ollama for local LLM inference. The alternative was calling `llama.cpp` directly, which would have meant building an HTTP shim around it. Ollama's REST API is mature enough that the trade wasn't close.

PyTorch + torchvision for image classification. CUDA 12.1 wheels are downloaded from PyTorch's index inside the Dockerfile so the host only needs the NVIDIA driver plus `nvidia-container-toolkit`.

3.7 Architecture

- **api-gateway** (port 8000) — decodes JWTs, injects `X-User-Id` / `X-User-Tier` / `X-User-Role` headers, proxies to the upstream services. Hosts the SocketIO server for chat and training presence.
- **auth-service** (port 8001) — user accounts, billing, admin operations, project and company ACL.
- **data-ingestion-service** (port 8002) — file uploads, SQL and S3 connectors, profiling, preprocessing, RAG ingestion, image-dataset extraction.
- **ml-training-service** (port 8003) — the fourteen tabular algorithms, model registry, RAG chat orchestration.
- **metrics-service** (port 8004) — TimescaleDB-backed hardware-telemetry collection.
- **dl-training-service** (port 8005) — PyTorch image classification.

A GitHub Actions workflow runs four jobs in parallel on every push: backend lint (`black`, `isort`, `flake8`), frontend lint (`eslint`, `tsc -noEmit`), backend tests (Postgres + Mongo + Redis spun up as service containers, `pytest` per microservice), and frontend tests (`vitest` with coverage upload). End-to-end CI runs in roughly six minutes for a frontend-only change, nine minutes for a backend change, and fourteen minutes when both sides move at once.

Conclusion

The methodology and toolchain held throughout the six sprints. Some choices — Flask in particular — are ones we would revisit if starting over, but they shipped a working system without rework. The next chapter opens the sprint walkthrough.

Chapter 4

Sprint 1: Authentication, Gateway, and Data Ingestion

Introduction

Sprint 1 was the foundation sprint. Nothing glamorous — no canvases, no models — but the design choices made in these two weeks shaped every later sprint. The gateway pattern and the shared-volume convention introduced here are the quiet load-bearing decisions of the whole project.

4.1 Sprint Backlog

Table 4.1: Sprint 1 backlog — foundation services.

ID	User Story / Task	Priority	Estimation (d)
US-01	Stateless JWT authentication with refresh-token rotation.	High	2
US-02	Redis-backed token revocation list.	High	1
US-03	Role-based access control (super-admin + tenant roles).	High	1
US-04	API gateway: strip client-supplied identity headers, inject from decoded JWT.	High	2
US-05	Optional TOTP two-factor authentication.	Medium	1.5
US-06	Per-IP and per-user rate limiting at the gateway.	Medium	0.5
US-07	Email verification on signup via MailHog.	Medium	1
US-08	Password reset round-trip via MailHog.	Medium	1
US-09	data-ingestion-service : CSV/Excel upload, async profiling.	High	2
US-10	SQL connectors (Postgres/MySQL) with Fernet-encrypted credentials.	Medium	1.5
US-11	Mongo-backed dataset metadata + Parquet preprocessing output.	High	1.5

4.1.1 Use case — Sign in with optional 2FA

Table 4.2: Textual description of the “Sign in with optional 2FA” use case.

Field	Value
Use case	Sign in with optional 2FA
Primary actor	End user (any tier)
Pre-conditions	Account exists and <code>email_verified = true</code> .
Trigger	User submits the Sign In form with email + password.
Nominal scenario	(1) Gateway forwards <code>POST /auth/login</code> . (2) Auth-service validates credentials with <code>bcrypt</code> . (3) If <code>totp_enabled = false</code> , issue access + refresh tokens. (4) Frontend persists access token in <code>sessionStorage</code> , sets <code>HttpOnly</code> refresh cookie. (5) Redirect to <code>/dashboard</code> .
Alt. scenario A	Credentials invalid → 401 + audit event <code>auth.login_failed</code> .
Alt. scenario B	<code>email_verified = false</code> → 403 with link hint to MailHog.
Alt. scenario C	<code>totp_enabled = true</code> → 200 <code>requires_2fa</code> + 5-min session token; user enters 6-digit code on <code>/2fa</code> .
Post-conditions	Refresh token JTI persisted; previous JTI blacklisted on next rotation.

4.1.2 Use case — Verify email via MailHog

Table 4.3: Textual description of the “Verify email” use case.

Field	Value
Use case	Verify email via MailHog
Primary actor	Newly registered user
Pre-conditions	<code>POST /auth/register</code> completed; token persisted as SHA-256 hash with 24-hour TTL.
Trigger	User clicks the link inside the verification email.
Nominal scenario	(1) Browser hits <code>/verify-email?token=...</code> (2) Frontend posts the token to <code>POST /auth/verify-email</code> . (3) Auth-service matches <code>sha256(token)</code> against <code>users.email_verification_token_hash</code> . (4) Sets <code>email_verified = true</code> ; clears the token row. (5) Frontend shows success card with a continue-to-sign-in button.
Alt. scenario A	Token expired ($\geq 24h$) → 400 <code>invalid_or_expired_token</code> ; user can request a fresh mail.
Alt. scenario B	Token reused → 400; the token hash is cleared on first success.
Post-conditions	Account is now eligible for sign-in.

4.2 Design

4.2.1 Why an opaque-but-decoded gateway

Two reasonable architectures sat on the table.

1. Have the gateway hit `auth-service` on every request to validate the JWT.
2. Have the gateway hold the same `JWT_SECRET_KEY` as `auth-service` and decode the token in-process.

The second option won, and we would make the same call again. The gateway is on the hot path; a synchronous HTTP round-trip per request adds latency that compounds badly under load. The price is that the gateway needs the same signing secret as the auth service — which, in a shared environment-file deployment, is a non-issue. A horizontally scaled production setup would need a key-distribution mechanism; for the demo target the in-process decode is the right trade.

4.2.2 Header-injection rules

The gateway strips any client-supplied `X-User-*`, `X-Company-*`, or `X-Project-*` header before forwarding the request, then re-adds those headers from the decoded JWT. This is the most important single security invariant in the system. Downstream services trust `X-User-Id` unconditionally, which means an attacker who can supply that header without traversing the gateway can impersonate anyone. The strip-then-inject pattern is deliberate, well tested, and not negotiable.

4.2.3 The shared-volume convention

The first instinct on the ingestion side was to have the API stream files to the worker over the network. That approach was built halfway before a simpler one became obvious: the Docker Compose stack already had a named volume (`uploaded_files`) that both the API container and the worker container could mount. Mounting it in both places collapsed the streaming-upload protocol into a one-line write — the API saves to `/uploads/<id>.csv`, the worker reads from the same path. That pattern then became the convention for every binary artefact in the system: trained models, RAG documents, and image-dataset extracts all live on named volumes mounted into whichever services need them.

4.2.4 Why MongoDB for dataset metadata

A dataset's profile is fundamentally schemaless. A CSV with two numeric columns produces a profile that looks nothing like the profile of a CSV with thirty mixed-type columns and a target. Postgres JSONB was considered for this storage and rejected. Mongo's document model is lossless, queryable enough for the use case, and imposes no schema migration when a new statistic is added in a later sprint.

4.3 Implementation

```
1 def _forward(upstream_base, path, require_auth=True, timeout=30):
2     if require_auth:
3         try:
4             verify_jwt_in_request()
5             claims = get_jwt()
6             g.user_id = get_jwt_identity()
7             g.user_role = claims.get("role", "")
8             g.user_tier = claims.get("tier", "free")
9         except Exception as e:
10            return jsonify({"error": "unauthorized", "message": str(e)
11            }), 401
12
13    # Strip every client-supplied identity header before forwarding.
14    _STRIP_PREFIXES = ("x-user-", "x-company-", "x-project-")
15    headers = {
16        k: v for k, v in request.headers
17        if k.lower() not in _HOP_BY_HOP
18        and k.lower() != "host"
19        and not k.lower().startswith(_STRIP_PREFIXES)
20    }
21    # Re-inject the authoritative ones from the decoded JWT.
22    if require_auth or hasattr(g, "user_id"):
23        headers.update(get_forwarded_headers())
24
25    resp = requests.request(
26        method=request.method, url=f"{upstream_base.rstrip('/')}{path}"
27        ,
28        headers=headers, data=request.get_data(), stream=True, timeout=
29        timeout,
30    )
31    return _stream_response(resp)
```

Listing 4.1: Gateway proxy with header injection (api-gateway/app/routes/proxy.py)

```
1 @celery.task(name="app.tasks.profiling.profile_dataset", bind=True)
2 def profile_dataset(self, dataset_id: str):
3     client = MongoClient(os.environ["MONGO_URL"])
4     db = client[os.environ.get("MONGO_DB", "nocode_ingestion")]
5     datasets = db["datasets"]
6     task_results = db["task_results"]
7
8     task_results.update_one(
9         {"task_id": self.request.id},
10        {"$set": {"status": "running", "progress_pct": 0}},
11        upsert=True,
12    )
13    doc = datasets.find_one({"dataset_id": dataset_id})
14    df = load_dataframe(doc["file_path"])
15
16    summary = compute_profile_summary(df, target_column=doc.get("
17    target_column"))
18
19    datasets.update_one(
20        {"dataset_id": dataset_id},
21        {"$set": {
```

```

21         "status": "ready",
22         "profiling_summary": summary,
23         "row_count": len(df),
24         "column_count": len(df.columns),
25     }},
26 )
27 task_results.update_one(
28     {"task_id": self.request.id},
29     {"$set": {"status": "success", "progress_pct": 100}},
30 )

```

Listing 4.2: Profiling task (data-ingestion-service/app/tasks/profiling.py)

4.3.1 Security posture

- Access tokens carry a fifteen-minute TTL. Refresh tokens live as `HttpOnly` cookies and rotate on every refresh; the previous token's JTI lands in the Redis blacklist so a stolen refresh token can actually be revoked.
- Passwords use `bcrypt` at cost 12. Refresh tokens themselves are stored as SHA-256 hashes in Postgres — the database-breach scenario yields a hash, not a usable token.
- TOTP-based two-factor authentication is optional per user; super-admins can require it per-tier from the admin panel.
- Per-IP and per-user rate limits sit at the gateway. The per-user limit defaults to 600req/min in development, deliberately loose because React 18 Strict Mode double-invokes `useEffect` and a tighter cap was producing spurious 429s. Production caps tighten to 120req/min.

4.3.2 SQL connectors and Fernet encryption

Users can configure a Postgres or MySQL connector inside the platform. Credentials are encrypted with a Fernet key (AES-128-CBC plus HMAC) held in the service's environment, then stored in Mongo. The same key is reused for any other secret the ingestion service needs to hold, and rotation is a documented operations procedure rather than a fire-fight. SQL queries themselves run through SQLAlchemy with parameter binding, which makes SQL injection structurally impossible rather than merely unlikely.

4.4 Unit Tests

The auth-service test suite covers more than 80% of the authentication path and the admin operations. Notable cases:

- Forged `X-User-Id` header: the request passes through the gateway, the forged header is stripped, the authoritative one from the JWT replaces it. Asserted on a synthetic upstream that echoes back the headers it saw.
- Refresh-token replay: a refresh token is used once, then replayed. The second use is rejected by the blacklist.

- Expired access token: the gateway returns 401 with the documented error envelope.
- Rate limit: 601 requests in one minute, the 601st returns 429 with a **Retry-After** header.

The ingestion suite uses `mongomock` so it does not need a real Mongo to run. It exercises the upload-then-profile path end-to-end with a small fixture CSV and asserts the profile shape.

Conclusion

By the end of Sprint 1 the platform had end-to-end authentication, a working refresh-token loop, a gateway pattern the next five sprints would build on, and an ingestion service that profiled arbitrarily messy CSVs asynchronously. The authoritative-headers convention became one of the project's most quietly useful design decisions — every subsequent service trusts **X-User-Id** without a second thought, because the gateway is the only path along which it can be set.

Chapter 5

Sprint 2: Visual Pipeline Editor and Tabular Machine Learning

Introduction

Sprint 2 was the largest single sprint of the project. It pulled together four pieces — the visual pipeline editor, the `ml-training-service` with its fourteen tabular algorithms, the live metric visualisations, and the post-training surface (SHAP explainability, model export) — into a coherent flow that a user could follow from canvas to artefact.

5.1 Sprint Backlog

Table 5.1: Sprint 2 backlog — canvas plus tabular ML stack.

ID	User Story / Task	Priority	Estimation (d)
US-12	React Flow canvas with custom node components and per-node validation.	High	3
US-13	Pipeline schema (Mongo document with nodes + edges).	High	0.5
US-14	<code>ml-training-service</code> hosting fourteen tabular algorithms.	High	4
US-15	<code>BaseMLModel</code> abstraction + algorithm-agnostic Celery task.	High	1
US-16	Model registry with versioning.	High	1
US-17	Live metric streaming over SocketIO.	Medium	1.5
US-18	SHAP explainability for supervised runs (TreeExplainer fast path).	Medium	1
US-19	Confusion matrices and residual plots.	Medium	0.5
US-20	Model export (joblib + generated FastAPI inference script).	Medium	1

5.2 CRISP-DM applied to the tabular ML sprint

We framed the ML work through the CRISP-DM lifecycle. The mapping makes it concrete what each phase delivered for this project.

Table 5.2: CRISP-DM phases mapped to Sprint 2 deliverables.

CRISP-DM phase	What this sprint delivered
Business understanding	Identify the supervised tasks a non-technical user can describe (classification, regression, clustering, forecasting). Surface fourteen named algorithms behind one shared canvas node.
Data understanding	Reuse the ingestion-service profile: missing values, skew, target imbalance, correlation. Surface alerts on the canvas's Dataset node.
Data preparation	Celery preprocessing task: 70/15/15 train/val/test split, column-wise imputation, scaling for distance-based algorithms, one-hot or target encoding by cardinality.
Modelling	<code>BaseMLModel</code> subclasses construct, fit, predict, and evaluate against a shared signature regardless of underlying library (sklearn, XGBoost, LightGBM, CatBoost, H2O, Prophet).
Evaluation	Accuracy/F1/AUC for classification, R^2 /MAE/RMSE for regression. F1 is preferred in the summary card when the target is imbalanced. SHAP feature importance for tree-based supervised runs.
Deployment	Versioned artefact in the registry; downloadable joblib + FastAPI launcher zip. Re-running a pipeline mints a new version rather than overwriting.

5.3 Design

5.3.1 Graph schema

A pipeline is stored in Mongo as a document with two arrays:

```

1 {
2   "pipeline_id": "...",
3   "user_id": "...",
4   "type": "ml" | "rag" | "dl",
5   "nodes": [
6     { "node_id": "n1", "type": "dataset", "position": {"x": 60, "y":
7       120},
8       "data": { "dataset_id": "..." } },
9     { "node_id": "n2", "type": "train", "position": {"x": 320, "y":
10      120},
11      "data": { "algorithm": "xgboost", "task_type": "classification",
12        "hyperparams": {...}, "target_column": "..." } },
13     { "node_id": "n3", "type": "evaluate", "position": {"x": 580, "y":
14      120},
15      "data": {} }
16   ],
17   "edges": [
18     { "source": "n1", "target": "n2" },
19     { "source": "n2", "target": "n3" }
20   ]
21 }
```

```

18 "status": "draft" | "running" | "done" | "error"
19 }

```

Listing 5.1: Pipeline document shape

5.3.2 The BaseMLModel abstraction

Fourteen algorithms with one orchestrator implies an abstraction. Rather than over-engineer it, we used a deliberately small abstract base class. Each algorithm subclass exposes `train`, `predict`, and `evaluate` with a shared signature, whether it wraps scikit-learn, XGBoost, LightGBM, CatBoost, or Prophet. The training Celery task is algorithm-agnostic: it constructs a model from a registry, trains it, asks for metrics, and serialises the estimator. Adding a new algorithm is, in practice, a one-file change.

```

1 class BaseMLModel(ABC):
2     def __init__(self, hyperparams: dict[str, Any]):
3         self.hyperparams = hyperparams
4         self._estimator = None
5
6     @abstractmethod
7     def train(self, X_train, y_train=None) -> None: ...
8     @abstractmethod
9     def predict(self, X) -> Any: ...
10    @abstractmethod
11    def evaluate(self, X_test, y_test=None) -> dict[str, Any]: ...
12
13    @property
14    def estimator(self):
15        return self._estimator

```

Listing 5.2: Algorithm-agnostic training core

The registry mapping algorithm strings to subclasses lives in `ml-training-service/app/services/`

5.3.3 Algorithm selection

The fourteen algorithms span four task families. The choice in each family was driven by the trade-off between training time, accuracy on typical tabular shapes, and how cleanly each library integrates with the abstraction above.

Classification and regression. For tree-based methods, XGBoost is the default because boosted trees consistently dominate on structured data with mixed numerical and categorical features — they capture non-linear feature interactions without explicit feature engineering, and the H2O implementation we use ships TreeSHAP for fast post-hoc explanation. Random Forest and Gradient Boosting Machines are exposed as alternatives for users who prefer simpler hyperparameter surfaces. Linear and logistic regression sit alongside the boosters as fast, interpretable baselines for small or very wide datasets where a tree’s overhead is not justified.

Clustering. K-Means is the default for its $O(nkd)$ cost and predictable behaviour; DBSCAN is exposed for data with non-spherical clusters; hierarchical agglomeration sits as a third option for small datasets where a dendrogram is more useful than a label.

Forecasting. Prophet is the default for time series with seasonal or holiday components, and it is one of the few algorithms where the cost of a compile-from-source

dependency (CmdStan) is genuinely worth paying. ARIMA is the alternative for series that are already stationary or short.

Heavy lifting on H2O. For XGBoost, GBM, and Random Forest, the platform uses H2O-3 in place of the scikit-learn implementations. H2O’s algorithms run faster on tabular data, parallelise across the host’s CPU cores by default, and expose feature importance plus SHAP values out of the box — which removed an entire subtask from the explainability work.

Table 5.3: Algorithm catalogue for the tabular ML mode.

Algorithm	Task	Backend	Why it is in the catalogue
Logistic Regression	Classification	scikit-learn	Fast linear baseline, interpretable coefficients.
Linear Regression	Regression	scikit-learn	Baseline for continuous targets, residual diagnostics.
Random Forest	Both	H2O-3	Robust default, no scaling needed, OOB error estimate.
XGBoost	Both	H2O-3	Highest accuracy on most tabular data with interactions.
LightGBM	Both	scikit-learn API	Faster than XGBoost on wide datasets, leaf-wise growth.
CatBoost	Both	scikit-learn API	Native categorical handling without one-hot blow-up.
GBM	Both	H2O-3	Mature gradient boosting, stable on small data.
K-Means	Clustering	scikit-learn	Default for spherical clusters, $O(nkd)$ cost.
DBSCAN	Clustering	scikit-learn	Density-based, handles non-spherical clusters.
Hierarchical	Clustering	scikit-learn	Dendrogram visualisation on small datasets.
KNN	Both	scikit-learn	Non-parametric baseline, useful for sanity checks.
SVM	Classification	scikit-learn	Maximum-margin classifier for small clean datasets.
Prophet	Forecasting	Meta Prophet	Time series with seasonality and holiday effects.
ARIMA	Forecasting	statsmodels	Stationary or already-detrended series.

5.4 Implementation

5.4.1 Frontend canvas

The canvas is a React Flow instance with custom node components. Each node type validates itself against the surrounding graph: the **Train** node shows an error badge if no **Dataset** node sits upstream, and the **Evaluate** node shows a warning if no training

run has produced a version yet. The validation function is pure — `(node, edges, allNodes) → {status, message}` — which makes it trivially testable. Errors render inline rather than as modal dialogs, so the user can fix three problems at once without clicking through three pop-ups.

5.4.2 Preprocessing and feature handling

Before a model sees a single row, the preprocessing Celery task splits the dataset into train / validation / test (default 70/15/15, configurable on the Train node), imputes missing values column-wise, scales numerical features when the algorithm requires it (logistic regression, K-Means), and encodes categorical columns using one-hot for low-cardinality columns and target encoding for high-cardinality ones. The transformer pipeline is stored alongside the model artefact so inference applies the same transformations to new rows.

5.4.3 Live training visualisation

Training emits per-step metric points over the SocketIO message queue. The `LiveTrainingChart` component subscribes to the pipeline's room (`pipeline_<id>`) on the `/training` namespace and plots the points on a Plotly scatter. The Y-axis is auto-scaled because progress percentage (0–100) and metric values (typically 0–1) live on different ranges — progress on the primary axis, metrics on the secondary. A small detail, but the chart was unreadable until that split landed.

5.4.4 Evaluation and metric interpretation

For classification runs, the platform reports accuracy, precision, recall, F1, and ROC-AUC when the task is binary or one-versus-rest multiclass. The validation report flags imbalanced targets (minority class below 20%) and prefers F1 over raw accuracy in the summary card, because accuracy on imbalanced data lies to the user. For regression, the report ships R^2 , mean absolute error, root mean squared error, and the residual distribution as a histogram — R^2 alone is misleading on heavy-tailed errors, and the histogram catches that visually.

The reference validation run uses a 250,000-row salary dataset with a Random Forest regressor and lands at an R^2 of 0.9785 in 4 min 12 s on the demo box. That score means roughly 97.85% of the salary variance is explained by the features — not a production-grade number on a real dataset where the residuals would need closer inspection, but a defensible result for a synthetic benchmark that exercises the full canvas-to-artefact path.

5.4.5 SHAP and the explainability surface

Every supervised training run computes SHAP values over a sampled subset of the test set. The bar chart of mean absolute SHAP values surfaces in the `Evaluate` node's results panel. SHAP (SHapley Additive exPlanations) attributes each prediction to a combination of feature contributions, drawing on Shapley values from cooperative game theory. For tree-based models — XGBoost, Random Forest, GBM, LightGBM, CatBoost — the platform uses the optimised `TreeExplainer` path, which is roughly two orders of magnitude faster than the general-purpose `KernelExplainer`. On a 5,000-row test set that is the difference between SHAP being a feature and SHAP being a wait.

5.4.6 Model export

The export step packages the trained `.joblib` artefact along with a generated FastAPI inference script. The user downloads a zip that they can drop onto any container runtime to serve the model without the platform at all. That property — being able to leave — mattered on principle. A no-code platform that locks a user in is not removing friction; it is relocating it.

5.4.7 Why joblib and not pickle

`joblib` was the deliberate choice for the tabular path because its custom pickler is optimised for objects containing large NumPy arrays — which every scikit-learn estimator holds — and can memory-map them on load. For a Random Forest of 500 trees the on-disk size difference was roughly $8\times$ in our measurements; the load-time difference was larger. For the deep-learning path in Sprint 4 we use `torch.save` on a `state_dict()` instead — a different ecosystem, a different convention.

5.5 Unit Tests

- Each `BaseMLModel` subclass has a happy-path test on a tiny fixture dataset (50 rows, four columns, a binary target). Assertions cover output shape and the presence of the documented metric keys.
- The algorithm-factory is exercised by parametrised tests — one per algorithm string — that construct the model, train, predict, and inspect the result envelope.
- Node validation: parametrised tests over (node-type, upstream-edges, expected-status) tuples.
- Export bundle: the generated zip is unzipped to a `tempdir` and the FastAPI script is imported to confirm it parses. The actual inference call is exercised by an end-to-end test in the deployment chapter.

Conclusion

Sprint 2 was the sprint where the project first felt like a real product. A user could open the canvas, drop three nodes, hit Run, and watch a model train in front of them. The `BaseMLModel` abstraction introduced here later absorbed the deep-learning extension in Sprint 4 without disturbing the orchestration code, and the SHAP plus export surface closed the loop between training a model and being able to do something with it.

Chapter 6

Sprint 3: Local Generative AI and Retrieval-Augmented Chat

Introduction

This was the most technically ambitious sprint of the project. A fully local Retrieval-Augmented Generation pipeline meant fitting an embedding model, a vector store, and an LLM inside the same Docker stack as the rest of the platform — and making the chat experience responsive enough that users would not notice they were running it on a consumer GPU.

6.1 Sprint Backlog

Table 6.1: Sprint 3 backlog — local RAG.

ID	User Story / Task	Priority	Estimation (d)
US-21	Document ingestion pipeline (PDF, DOCX, plain text).	High	1
US-22	Local embedding via <code>sentence-transformers</code> .	High	0.5
US-23	Vector store in <code>pgvector</code> (IVFFlat index).	High	1
US-24	Ollama integration serving Llama 3.2 3B Q4_K_M.	High	1
US-25	Chat orchestration — embed, retrieve top- <i>k</i> , build prompt, stream.	High	2
US-26	Streamed UI: tokens render as they arrive over SocketIO.	High	1.5
US-27	RAG-mode toggle on the canvas with three node types.	Medium	1
US-28	Source-chunks chip showing retrieved chunks + similarity.	Low	0.5

6.2 Concepts fondamentaux

Three concepts carry the weight of this sprint; the design and implementation sections below assume them.

Embeddings. A sentence-transformer maps a chunk of text to a fixed-dimensional vector (384 dimensions for MiniLM-L6-v2) such that cosine similarity of the vectors correlates with semantic similarity of the text. The embedding is one-way and lossy; the vector cannot be inverted back into the chunk.

Vector store with approximate nearest-neighbour search. pgvector's IVF-Flat index partitions the vector space into clusters at index-build time, then at query time visits the closest clusters only. For the local-tenant scale this project targets, the recall/latency trade-off lands firmly on the latency side — under 50 ms for top- k over a few hundred thousand chunks.

Retrieval-Augmented Generation. Rather than fine-tuning a model on the user's documents, the pipeline retrieves the top- k most similar chunks at query time and concatenates them into the prompt as context. The model is instructed to answer from the provided context and to refuse rather than guess. This trades a training step for a retrieval step — the right trade when the corpus changes faster than a fine-tune can keep up with.

6.3 Design

A Retrieval-Augmented Generation pipeline grounds a language model in documents the user provides. The pipeline embeds each document chunk into a vector, stores those vectors in a database that supports similarity search, and at query time embeds the question, retrieves the most similar chunks, and concatenates them into the model's prompt as context. The model then answers from the retrieved content, which keeps responses grounded in the source documents and sidesteps the hallucination patterns that pure-LLM chat is prone to.

The component picks for this project:

- **sentence-transformers/all-MiniLM-L6-v2** for embeddings. The model is small (around 22 MB on disk, 384 output dimensions), runs fast on CPU, and produces embeddings whose cosine similarity correlates well with semantic similarity in English and reasonably well in French. Larger models (**all-mpnet-base-v2**, OpenAI's **text-embedding-3-small**) would lift retrieval quality a few points but cost CPU latency and, in the OpenAI case, violate the data-sovereignty constraint outright.
- **pgvector** for the vector store, with an IVFFlat index. Co-locating the store in the auth-service Postgres instance avoided spinning up a sixth backing service. IVFFlat trades a small amount of recall for a large gain in query latency, which is the right trade for a few hundred thousand chunks per tenant.
- **Ollama running llama3.2:3b** as the LLM engine. Q4_K_M quantisation fits in roughly 2 GB of VRAM, which leaves headroom on the 6 GB demo GPU for embeddings, canvas, and the training queue to share the same device without stepping on each other.

6.4 Implementation

```
1 @celery.task(name="app.tasks.rag_ingest.process_rag_document", bind=
   True)
2 def process_rag_document(self, document_id, pipeline_id, file_path):
3     from langchain_text_splitters import RecursiveCharacterTextSplitter
4     from langchain_community.document_loaders import PyPDFLoader
5
6     docs = PyPDFLoader(file_path).load()
7     splitter = RecursiveCharacterTextSplitter(
8         chunk_size=500, chunk_overlap=50, length_function=len,
9     )
10    split_docs = splitter.split_documents(docs)
11    chunk_texts = [d.page_content for d in split_docs if d.page_content
12                  .strip()]
13
14    # Local embedding via sentence-transformers -- no network call.
15    vectors = embed_texts(chunk_texts)
16    insert_chunks(
17        pipeline_id=pipeline_id,
18        document_id=document_id,
19        chunks=chunk_texts,
20        embeddings=vectors,
```

Listing 6.1: Document ingestion task
(ml-training-service/app/tasks/rag_ingest.py)

6.4.1 Streamed chat

The chat response streams token by token over SocketIO. The `ml-training-service` acts as producer: it embeds the question, queries pgvector for the top- k chunks (default $k = 4$), builds the prompt with the retrieved context, and forwards Ollama's `stream=true` chunks straight into the `/chat/<pipeline_id>` room. The frontend appends each token to the current assistant message as it arrives. The first time that worked end-to-end — on a laptop, with no network involved — was the moment the project's thesis stopped feeling speculative.

6.4.2 Chunk sizing trade-offs

Five hundred characters with a fifty-character overlap is the default chunking. We tested shorter chunks (256, 128) and longer ones (1024). Short chunks raised retrieval precision on single-fact questions but ruined it for anything that needed multi-sentence context; long chunks did the opposite, packing too much unrelated material into each retrieval slot. Five hundred sat in the middle. It is a defensible default, not a universal answer; the backlog notes a follow-up to expose chunk size as a per-pipeline setting once the platform has enough usage data to choose by document type automatically.

6.4.3 Prompt construction

The system prompt instructs the model to answer from the retrieved context and to refuse rather than guess if the context does not cover the question. The retrieved chunks

are inserted in similarity order, each prefixed with a document identifier so the model can cite. The user-facing chip “ n sources used” lists the chunks and their cosine similarity scores, which gives the reader a way to verify the answer against the original source — the closest thing to accountability that a generative system can offer.

6.5 Unit Tests

- Chunking determinism — the same input file produces the same chunks across runs.
- Embedding shape — 384 dimensions, normalised.
- Retrieval top- k on a deterministic five-chunk corpus with hand-labelled relevance.
- Chat orchestrator: Ollama is mocked at the HTTP boundary, the orchestrator is asserted to emit the right SocketIO events in the right order.

The Ollama process itself is not exercised in unit tests — it sits behind the smoke-test script in the deployment chapter.

Conclusion

Sprint 3 proved the local-generative-AI thesis. The chat is not as fast as GPT-4 over the wire, and it never will be on this hardware, but it is interactive enough to be useful and it keeps the user’s documents on the user’s machine. For the regulated audiences this project is framed against, that trade-off is the right one.

Chapter 7

Sprint 4: Deep Learning for Image Classification

Introduction

This sprint added the third pipeline mode. Deep learning on the demo hardware is a constrained problem — 6 GB of VRAM does not forgive carelessness — so the design started from the constraint and worked backwards. The result is a small, opinionated catalogue rather than a kitchen-sink layer builder, and the opinionation is the feature.

7.1 Sprint Backlog

Table 7.1: Sprint 4 backlog — deep learning for image classification.

ID	User Story / Task	Priority	Estimation (d)
US-29	New microservice: <code>dl-training-service</code> .	High	1
US-30	Three CNN architectures (LeNet, ResNet-9, MobileNet-V3-Small).	High	2
US-31	Image-dataset ingestion (zip of <code><class>/<file></code>).	High	1
US-32	Static VRAM estimator (the VRAM guard).	High	2
US-33	Tier-aware ceilings on epochs and batch size.	High	0.5
US-34	Live training visualisation (reused from Sprint 2).	Medium	0.5
US-35	Inline single-image inference (the “Try it” panel).	Medium	1
US-36	DL-mode canvas toggle with three new node components.	Medium	1.5

7.2 Concepts fondamentaux

Convolutional layers. A 2D convolution slides a small trainable kernel across the input image, producing a feature map that responds to local patterns (edges, textures, shapes). Stacking several convolution + pooling layers builds a hierarchy from low-level features to class-discriminative ones.

Residual connections. A residual block adds the block’s input to its output, so a layer learns the residual mapping $F(x) + x$ rather than $F(x)$ alone. This eases gradient flow in deeper networks and is the reason the custom ResNet-9 trains predictably even on small datasets.

Transfer learning. Loading a model with weights already trained on ImageNet, then fine-tuning the final layers on the user’s dataset. For 200-image custom datasets the transfer-learned MobileNet-V3 reaches usable accuracy in minutes; a random-init run on the same data takes an order of magnitude longer.

Batch normalisation and the optimiser memory footprint. Adam keeps two moment estimates per parameter (mean and variance), which doubles the optimiser-state memory compared to plain SGD. This factor is the dominant term in the VRAM guard’s static estimate documented below.

7.3 Design

7.3.1 Scope decisions

What stayed in:

- Three architectures: LeNet (smoke-test scale), a custom ResNet-9 (*tiny_resnet*), and torchvision’s MobileNet-V3-Small with optional ImageNet transfer learning.
- Image-dataset upload: a zip of `<class>/<file>` folders, extracted into the canonical `ImageFolder` layout that PyTorch’s data loaders expect.
- Live training visualisation reused from the tabular path (the `training_epoch` socket event fans out into train/val loss and val-accuracy series).
- Single-image inference (the “Try it” panel) inline immediately after training completes.

What stayed out, deliberately:

- Freeform layer-builder DAG — too easy for the user to configure something that does not fit.
- LLM fine-tuning — won’t fit inside 6 GB.
- Object detection and segmentation — different label format, different demo, different scope.
- ONNX export — a one-file follow-up if needed, not in scope.
- Multi-GPU and distributed training.

Drawing this line clearly mattered as much as anything we built. A no-code deep-learning interface that lets the user wire up an architecture that doesn't fit is, in practice, a debugger the user did not ask for.

7.3.2 Why CNN architectures, and which three

The image-classification problem on small custom datasets (hundreds, not millions, of labelled images) favours convolutional networks over transformer-based vision models. Vision Transformers need orders of magnitude more data or aggressive pretraining to match a competent CNN's accuracy, and the demo budget does not support either. The three architectures in the catalogue were chosen to span three operating points.

LeNet (roughly 60 k parameters) is the smoke-test architecture. It trains in seconds on Fashion-MNIST-class data and serves as the smallest credible CNN — useful for verifying the training path end-to-end before committing to a longer run.

Tiny ResNet (a custom ResNet-9, roughly 2.7 M parameters) is the workhorse. Two residual blocks with channels $64 \rightarrow 128 \rightarrow 256$, skip connections that ease gradient flow, and batch normalisation throughout. The architecture reaches around 90% accuracy on CIFAR-10 in five epochs at batch 64, input 64 px — a defensible accuracy/time trade-off for an on-stage demo.

MobileNet-V3-Small (roughly 2.5 M parameters) is the transfer-learning option. Loaded from torchvision with ImageNet weights, the architecture's depthwise-separable convolutions keep the inference cost low while the pretrained features carry most of the work on small custom datasets. A fresh 200-image dataset can reach 80%+ accuracy in two minutes; random-init would take twenty.

Table 7.2: CNN architecture catalogue with operating envelope.

Architecture	Params	Demo VRAM	Operating point
LeNet	~60 k	<0.3 GB	Smoke-test scale. Confirms the training path end-to-end before committing to a longer run.
Tiny ResNet-9	~2.7 M	~1.5 GB at 64 px batch 32	Workhorse. Hits ~90% on CIFAR-10 in five epochs; the demo default.
MobileNet-V3-Small	~2.5 M	~2.0 GB at 224 px batch 16	Transfer learning from ImageNet. Reaches 80%+ on a 200-image custom dataset in two minutes.

7.3.3 Why a separate microservice

Two reasons.

First, `torch` plus `torchvision` add about 2 GB of dependencies that the existing H2O-based `ml-training-service` does not need. Co-locating them would have doubled the size of an image that rebuilds on every CI run.

Second, GPU access is per-container in Docker. The DL service declares a `deploy.resources.reserved` block; the H2O service runs on CPU. Mixing them in one container would either deny GPU to the DL workload or force the H2O workload onto a GPU it cannot use.

The service has its own Celery queue (`dl_training`) so a 20-epoch CIFAR run cannot starve the fast tabular queue.

7.4 Implementation

7.4.1 Preprocessing and augmentation

Image tensors are resized to the architecture’s expected input, normalised to ImageNet statistics when transfer learning is enabled, and augmented at train time with random horizontal flips and small rotations. Augmentation is intentionally conservative: heavier recipes (CutMix, MixUp) help on large datasets and hurt on small ones, and small datasets are the common case here.

7.4.2 The VRAM guard

The most subtle piece of Sprint 4 is the static memory estimator:

```
1 def estimate(arch: str, input_size: int, batch_size: int) -> Estimate:
2     weights = params_mb(arch)
3     optimizer = 3.0 * weights # weights + grads + 2 Adam
4     moments
5     activations = (
6         _ACTIVATION_PER_PIXEL_MB[arch]
7         * (input_size * input_size)
8         * batch_size
9     )
10    total = weights + optimizer + activations + _CUDA_RUNTIME_MB
11    return Estimate(weights_mb=weights, optimizer_mb=optimizer,
12                    activations_mb=activations,
13                    runtime_mb=_CUDA_RUNTIME_MB, total_mb=total)
```

Listing 7.1: Static VRAM estimate
(`dl-training-service/app/services/vram_guard.py`)

The route refuses any `/dl/train` request whose estimate exceeds the user’s tier-resolved budget minus a 1 GB headroom for the CUDA runtime and allocator fragmentation. This is the single most important demo-safety net in the system: a wrong batch-size press cannot OOM the GPU on stage.

The constants in `_ACTIVATION_PER_PIXEL_MB` are conservative empirical estimates. The unit-test suite enforces that every `(arch, input_size, batch_size)` triple in the documented catalogue fits on a 6 GB card after headroom, but it does not pin the absolute numbers — those drift slightly between GPU generations and need recalibration on first deploy.

7.4.3 Tier-aware ceilings

Sprint 4 added two per-tier columns to the `subscriptions` table: `max_dl_epochs` and `max_dl_batch_size`. The defaults: Free at 5 epochs / batch 32, Solo at 20 epochs / batch 64, Company at 50 epochs / batch 64.

The defence-in-depth chain runs:

1. The frontend's `DLTrainNode` reads the user's `limits` from `/users/me` and clamps the slider `max` accordingly — a free-tier user literally cannot select 20 epochs from the UI.
2. The route enforces the tier ceiling regardless: a custom client posting 50 epochs to `/dl/train` on a free-tier token receives `400 epochs_over_tier_limit`.
3. The `vram_guard` then checks the static memory estimate against the tier's VRAM budget.
4. The service-wide `HARD_MAX_EPOCHS` and `HARD_MAX_BATCH_SIZE` caps in `config.py` sit as an absolute ceiling that no tier override can lift.

Four layers may sound paranoid, but each one catches a different class of mistake — accidental, malicious, oversized, pathological — and none of them is expensive.

7.4.4 Frontend integration

The canvas gained a third toggle button (“Deep Learning”), three new node components (`ImageDatasetNode`, `CNNArchNode`, `DLTrainNode`), and a fourth preset (`dlStarter`). The three-family lock logic was extended to lock all three families against each other once any node of a given family lands on the canvas.

The `ImageDatasetNode` body shows class chips, total-image count, and a deterministically seeded thumbnail strip (three images per class, pulled from `/datasets/<id>/image-preview` as base64-encoded JPEGs). The `CNNArchNode` surfaces a `pretrained` toggle that disables itself when the chosen architecture has no canonical ImageNet checkpoint — the user sees *why* the option is unavailable rather than having a control silently disappear, which is the small UX detail that decides whether people trust the interface.

7.4.5 Live progress and inference

The training Celery task emits a `training_epoch` socket event per epoch with `{epoch, train_loss, val_loss, val_acc}`. The `LiveTrainingChart` component fans that single event into three Plotly series sharing the same epoch-indexed X-axis.

When training completes, a `DLPredictPanel` component renders inline immediately under the success alert. The user drops an image, the panel POSTs it to `/dl/predict/<version_id>`, and the top-five softmax probabilities render as ranked horizontal bars.

7.4.6 Demo path end-to-end

For the live demo, the path is:

1. `python scripts/seed_demo_image_dataset.py /tmp/demo.zip` — generates a 30-image, 3-class synthetic dataset (red circles, green squares, blue triangles).
2. Upload the zip from the Data page.
3. Drop the `dlStarter` preset onto a fresh pipeline.
4. Click Run. Roughly 30 seconds later, the predict panel renders.

5. Drop an image — one of the synthetic ones, or a real one from a phone — and inference returns in roughly 5 ms on CPU.

The reference DL validation run uses `tiny_resnet` at 64px, batch 32, five epochs against the synthetic dataset. Terminal validation accuracy lands at 100% in roughly 30 seconds on a 1660 Super — a number that is honest about the dataset’s simplicity (three visually distinct classes) but proves the full pipeline.

A separate `scripts/dl_smoke.sh` script exercises the entire chain non-interactively: gateway reachability, GPU visibility, the oversize-refusal path, the demo run, and the polling loop. The script has caught regressions in seemingly unrelated services more than once.

7.5 Unit Tests

The `dl-training-service` ships 22 unit tests:

- Architecture forward-pass shapes — one test per architecture on a known input tensor.
- VRAM-guard envelope — parametrised over every documented (`arch`, `input_size`, `batch_size`) triple, asserts the estimate sits under the 6 GB-minus-1 GB budget.
- Oversize refusal — a request with an obviously oversized batch is rejected by the route before training starts.
- End-to-end `build` → `save` → `predict` on a tiny LeNet against a 12-image fixture dataset, against a real torch instance (CPU on CI).
- Tier-ceiling enforcement — a free-tier token posting 20 epochs is rejected with the documented error code.

Conclusion

Sprint 4 was the riskiest sprint of the project. Bolting an entirely new workload onto a platform that had calmed down into three sprints of mostly-tabular and RAG work could have destabilised the whole thing. It did not, mostly because the conventions established in earlier sprints — the shared-volume pattern, the Celery-per-service split, the gateway header injection — absorbed the new service without bending.

Chapter 8

Sprint 5: Multi-tenancy, Project ACL, and Real-Time Collaboration

Introduction

Up to this point the platform had been a single-user experience. The company-tier story needed something else: multiple people on the same pipeline at the same time, with sensible access controls and enough real-time feedback that they would not accidentally undo each other's work.

8.1 Sprint Backlog

Table 8.1: Sprint 5 backlog — multi-tenancy and collaboration.

ID	User Story / Task	Priority	Estimation (d)
US-37	Per-pipeline access control with five distinct roles.	High	1.5
US-38	Server-side and client-side role enforcement.	High	1
US-39	Real-time canvas presence (cursors and node highlights).	High	2
US-40	Threaded chat per pipeline.	Medium	1.5
US-41	One-click Google Meet links via Google OAuth.	Medium	1
US-42	Per-pipeline notification fan-out.	Medium	0.5

8.2 Concepts fondamentaux

SocketIO rooms. A room is a server-side label that groups client connections; emitting to a room fans the message out to every connection joined to that label. The collaboration

layer joins each client to the room `pipeline_<id>` the moment the pipeline opens, and unjoins on unmount.

Throttling for real-time presence. Cursor moves can fire at 60 Hz; broadcasting every event saturates the message bus and the clients without adding signal. The frontend throttles to one event per 50 ms, which is below the human perception threshold for cursor lag and an order of magnitude lighter on the network.

Authoritative role resolution at the gateway. The five roles (Owner, Project Manager, Data Scientist, Analyst, Viewer) are resolved at the gateway and stamped onto `X-Project-Role` just like `X-User-Id`. Downstream services trust the header without re-checking, which keeps the zero-trust boundary single-layered.

8.3 Design

8.3.1 The role hierarchy

- **Owner** — the user who created the pipeline. Cannot be removed.
- **Project Manager** — can invite, can edit, cannot remove the owner.
- **Data Scientist** — can edit nodes, can run training, cannot manage members.
- **Analyst** — can view metrics and download artefacts, cannot run training.
- **Viewer** — read-only.

The role is enforced both server-side and client-side. The gateway resolves the user's role for the project on every `/pipelines/*` request and stamps `X-Project-Role`; the frontend disables UI affordances based on the same role. Server-side enforcement is authoritative; the client-side disable is UX, not security. Mixing those up is one of the easier mistakes in this area.

8.4 Implementation

8.4.1 Real-time presence

When a user opens a pipeline, the gateway joins them to the `pipeline_<id>` SocketIO room. Cursor positions and node selections are emitted as throttled (50 ms) events. The frontend maintains a per-user coloured cursor overlay on the canvas. Watching another user's cursor move across your screen is a small touch that makes the platform feel inhabited in a way that chat alone never quite does.

8.4.2 Pipeline chat and Google Meet

Each pipeline gets its own threaded chat, persisted to Postgres. An “@” mention fires a notification (which the Sprint 6 bell surfaces). The Google Meet integration relies on Google's OAuth flow — not for data-sharing reasons, but because a Meet link generated server-side without OAuth is not tied to the inviting user's calendar, which makes the link useless for the actual workflow.

8.4.3 Why the gateway resolves the role

The role-resolution lookup happens inside the gateway rather than in each downstream service. Two reasons.

First, the gateway already maintains the request-scoped user identity. Adding role resolution here keeps the zero-trust pattern intact — downstream services trust the stamped `X-Project-Role` the same way they trust `X-User-Id`.

Second, it centralises the cache. A short-lived Redis-backed cache on `(user_id, pipeline_id) → role` cuts the auth-service lookup count to a small fraction of the request count, without spreading caching logic across four services.

8.5 Unit Tests

- Role-matrix table-test: for each (role, action) pair, the expected permit/deny outcome.
- Role demotion mid-session: a Project Manager has their role downgraded to Viewer while connected; the next canvas-edit event is rejected.
- Owner immutability: an attempt to remove the owner returns the documented error code.
- SocketIO presence: two test clients join the same pipeline room and assert that each sees the other's cursor events.

Conclusion

The collaboration layer turned the platform from a personal tool into something a team could share. The interesting part of this sprint was how little of the existing architecture had to change to support it — the gateway already injected user identity, the SocketIO bus was already in place for live training, and the role check fit naturally into the existing header convention.

Chapter 9

Sprint 6: Dashboard, i18n, Accessibility, and Admin Console

Introduction

Every project has the sprint where the punch list of small things finally gets crossed off. This was that sprint. The dashboard, the billing UI, internationalisation, the high-contrast theme, the notification bell, the admin operations dashboard, user impersonation, and undo/redo on the canvas — each modest on its own, but together they moved the platform from “demonstrably works” to “demonstrably operable”.

9.1 Sprint Backlog

Table 9.1: Sprint 6 backlog — operability and polish.

ID	User Story / Task	Priority	Estimation (d)
US-43	User-facing dashboard (datasets, pipelines, model versions).	High	1.5
US-44	Billing UI (free / solo / company tier; Stripe checkout).	High	2
US-45	Internationalisation (English + French, generated bindings).	High	1
US-46	High-contrast theme + WCAG AA audit.	Medium	1
US-47	Persistent notification bell.	Medium	1
US-48	Admin operations dashboard (5 panels).	High	2
US-49	User impersonation with hard five-minute TTL.	High	1
US-50	Undo / redo on the canvas with drag-coalescing.	Medium	1

9.2 Design

9.3 Implementation

9.3.1 The user dashboard and billing UI

The dashboard is a single page with three tabs (Datasets, Pipelines, Models), each backed by a paginated table with server-side sorting. The right rail shows the user's quota (datasets, model versions, RAG documents) for the current tier plus a “Manage Plan” link to the billing UI. Billing itself goes through Stripe Checkout for purchases and the Stripe Customer Portal for upgrades and cancellations — the platform never handles a card number directly.

9.3.2 Internationalisation

Every user-facing string moved to `i18next`, with English and French resource bundles. TypeScript bindings are generated from the English bundle, so a missing French key fails the type-check rather than rendering as a raw key string in production. That small piece of automation paid for itself the first time someone forgot to translate a label.

9.3.3 High-contrast theme and accessibility

A high-contrast theme was added for users with visual impairments, toggleable from the navbar. Every interactive element gained an `aria-label`; the colour palette was checked against WCAG AA contrast ratios. Accessibility is not an optional extra at the company tier — in some jurisdictions it is a procurement requirement.

9.3.4 Notification bell with persistence

A bell icon in the navbar shows unread chat mentions, training-done events, training-failed events, document-indexed events, and meeting links. The store is persisted to `localStorage` so a page refresh does not drop the unread count, which sounds obvious until the first version is built and the count keeps resetting to zero.

9.3.5 Admin operations dashboard

The super-admin role gained a dedicated page with five panels:

- **User management** — search, suspend, delete, override quotas.
- **Audit log viewer** — paginated table of every sensitive action (login, password reset, impersonation, suspension), with IP and JSON detail.
- **Queue monitor** — live stats from each Celery queue.
- **Hardware monitor** — CPU, RAM, GPU utilisation sampled from the metrics-service every five seconds.
- **Migration drift panel** — compares each service's latest Alembic revision against the database and surfaces the diff.

9.3.6 User impersonation

Super-admins can “view as” any user from the user-management table. The auth-service mints a separate five-minute access token with an `imp_actor` claim identifying the original super-admin. A red banner persists across every page during the impersonation session, showing the target email and a live countdown to expiry. Clicking **Exit** (or hitting Logout in the navbar) restores the super-admin’s original token. Two audit events are recorded: `admin.impersonate_start` and `admin.impersonate_end`.

The hard TTL is the load-bearing piece. An impersonation session that does not time out is, sooner or later, an incident waiting to happen.

9.3.7 Undo / redo on the canvas

A 50-snapshot history stack with drag-coalescing (a single drag gesture produces one snapshot, not 60) and `Cmd/Ctrl+Z` plus `Cmd/Ctrl+Shift+Z` keyboard shortcuts. Two `IconButton`s sit next to the auto-layout button for users who do not know the keyboard shortcut. A small feature, but one of those things nobody notices until it is missing.

9.3.8 Cross-cutting changes

Two architectural shifts landed this sprint that do not fit neatly under any one feature.

First, auth state moved to `sessionStorage`. Before this sprint, refreshing the page during impersonation lost the original super-admin token — the Zustand auth store was in-memory only. Wrapping it in `persist(..., sessionStorage)` fixed the impersonation case and quietly improved refresh behaviour for normal users at the same time.

Second, destructive admin actions now require a typed confirmation. Delete-user and delete-announcement modals demand the user retype the target’s email or title before the action button enables. The native `confirm()` dialog was too easy to misclick during a live demo, and once was enough.

9.4 Unit Tests

- i18n key parity: a CI step diffs the English and French bundles and fails if a key is missing on either side.
- Notification reducer: state transitions for each event type (mention, training-done, etc.) plus a persistence round-trip.
- Impersonation flow: the original token, the impersonation token, and the post-exit restore are asserted across three pytest cases.
- Audit-log presence: every sensitive route is exercised and the audit table is asserted non-empty for that action.
- Undo / redo: a sequence of canvas edits, then a fixed number of `Ctrl+Z` presses, then a deep-equality check against an earlier snapshot.

Conclusion

The punch-list sprint never makes for thrilling reading, but it is where the platform earned the right to be presented to a real user. The admin console in particular changed how we think about the platform — having a single place to inspect queue health, hardware load, and migration state turned operations from a vague worry into a tractable one.

Chapter 10

Testing and Deployment

Introduction

Testing and deployment evolved in parallel with the rest of the system. By the end they looked nothing like what we had at the start. This chapter covers both — how the tests are organised, what they actually verify, and how the whole thing ships.

10.1 Test Strategy

10.1.1 Backend Testing

The test suite is split by service. Each Python service has a `tests/` directory with `pytest` fixtures that fake the backing services (Mongo via `mongomock`, Celery via a monkey-patched `apply_async`) so the suite runs in seconds on a CI runner without spinning up the real stack.

Coverage is unevenly distributed, which is worth being honest about. Table 10.1 summarises the load-bearing paths per service.

Table 10.1: Backend test coverage by service.

Service	Line cov.	Load-bearing path tested
auth-service	>80%	Login, refresh rotation, TOTP, email verification, password reset, admin ops, impersonation TTL.
api-gateway	~70%	Header strip-then-inject invariant, rate limit, public-prefix routing.
data-ingestion-service	~65%	Upload-then-profile end to end (mongomock), SQL probe, Fernet round-trip.
ml-training-service	~60%	Algorithm factory parametrised across 14 algorithms; SocketIO streaming path not exercised.
dl-training-service	22 cases	Forward-pass shapes per arch, VRAM-guard envelope, oversize refusal, end-to-end LeNet train→save→predict on CPU.

10.1.2 Frontend Testing

The frontend uses `vitest` for unit tests but relies mostly on TypeScript strict mode plus ESLint as the structural guarantee. In practice more than 99% of regressions in this codebase are caught by `tsc -noEmit` before they ever reach a runner. Typing is the cheapest test you can write.

10.1.3 CI runs

End-to-end CI runs in roughly six minutes for a frontend-only change, nine minutes for a backend change, and fourteen minutes when both sides move at once.

10.2 Deployment

The whole stack ships as a single `docker-compose.yml` with 14 services, summarised in Table 10.2.

Table 10.2: Deployment topology — 14 containers, three named volumes.

Container	Port	Role
api-gateway	8000	JWT decode + header injection + SocketIO.
auth-service	8001	Accounts, ACL, billing, admin operations.
data-ingestion-service	8002	File upload, SQL connectors, profiling, RAG ingestion.
ml-training-service	8003	14 tabular algorithms + RAG chat orchestration.
metrics-service	8004	Hardware telemetry into TimescaleDB.
dl-training-service	8005	PyTorch image classification (GPU).
3 × Celery workers	internal	One per CPU-bound service.
postgres (pgvector)	internal	Auth, ACL, vector store.
mongo	internal	Schemaless data (datasets, pipelines, results).
redis	internal	Celery broker / result / SocketIO bus.
timescaledb	internal	Hardware telemetry time series.
ollama	internal	Local LLM inference (Llama 3.2 3B).
mailhog	1025 / 8025	Dev SMTP sink + web UI for verification/reset mails.

A `make up` brings the entire platform online. First-run duration is dominated by model pulls: Ollama fetches the LLM weights on first boot (around 2 GB), and the dl-training service pulls the CUDA wheels on first build (around 1 GB). After that, `make up` is a matter of seconds.

10.2.1 Volumes

Three named Docker volumes carry the binary artefacts that survive container restarts: `uploaded_files` (datasets and image zips), `model_artifacts` (trained models), and `ollama_models` (LLM weights). `uploaded_files` is mounted into the gateway, both ingestion containers (API and worker), the ml-training containers, and the dl-training containers — the shared-volume convention from Sprint 1 ended up applied very broadly, arguably more broadly than originally planned.

10.2.2 Healthchecks and dependency ordering

Every backing service has a `healthcheck`, and every application service `depends_on` the relevant healthchecks with `condition: service_healthy`. The first time the stack was deployed without this discipline, the auth service started before Postgres had finished initialising and crash-looped on the Alembic migration. The healthcheck convention has been non-negotiable for any service that touches a database ever since.

10.2.3 Network and security topology

The gateway is the only container exposed on the host. All other services bind to the internal Docker network. The Postgres instance is not reachable from outside the Compose stack; the only path to it is through the auth-service or the ml-training-service, and the

only path to either of those is through the gateway. That is the same zero-trust posture as the JWT header injection, applied at the network layer.

10.2.4 Bugs encountered during deployment

A non-exhaustive list, in roughly the order they bit us:

- **Prophet missing CmdStan.** Prophet’s probabilistic backend is a C++ binary that has to be compiled at image-build time, not pip-installed. The fix was a `python -c "import cmdstanpy; cmdstanpy.install_cmdstan()"` step in the ml-training Dockerfile.
- **Flask CLI app discovery inside containers.** Running `flask db migrate` via `docker compose exec` could not find the app factory; the fix was an explicit `-e FLASK_APP=app.main` on every CLI invocation, baked into the Makefile so it would not be forgotten.
- **React 18 Strict Mode and rate limiting.** Strict Mode double-invokes `useEffect` in development; the rate limit was tripping. Bumped the per-user limit to 600/min in development, kept the production cap tight.
- **torch CUDA wheels in CI.** The `dl-training-service` Dockerfile pulls CUDA-12.1 torch from PyTorch’s index; on a CI runner that would have been a 1 GB download for no benefit. CI installs CPU-only torch wheels separately, gated on `matrix.service == 'dl-training-service'`.
- **Ollama GPU detection in Compose.** The default compose deployment runs Ollama on CPU. Enabling the `deploy.resources.reservations.devices` block requires `nvidia-container-toolkit` on the host, which is not the default on Ubuntu Server. Documented in the README and commented out in the compose file by default.
- **DL VRAM-guard calibration.** The static per-pixel constants were tuned against documented Turing memory numbers rather than measured ones. The first real GPU run on the 1660 Super exceeded the estimate by about 15% on MobileNet at 224px batch 32; the constants were nudged upwards accordingly. The unit-test suite tracks the envelope shape, not the absolute MB, so this calibration drift was caught empirically rather than by a failing test.

Conclusion

The stack is, by deployment-engineering standards, modest — one host, fourteen containers, make up. That simplicity is the point. A platform whose deployment story requires Kubernetes expertise would not be a credible answer to the question this project set out to ask.

General Conclusion

The end-of-studies project set out to build a privacy-first, no-code AI platform that could stand alongside cloud-managed ML services without making the data-sovereignty compromises those services demand. After six sprints, the platform supports three pipeline modes — tabular ML with fourteen algorithms, fully-local RAG-augmented chat, and image-classification deep learning — runs on a single consumer machine with 6 GB of VRAM, and exposes that capability through a visual canvas designed for users who do not write Python.

Key deliverables

- **Five Python microservices** totalling roughly 6,500 lines of code across the api-gateway, auth-service, data-ingestion-service, ml-training-service, metrics-service, and dl-training-service. Three independent Celery worker processes handle the asynchronous workloads.
- **A React + TypeScript single-page application** of roughly 13,000 lines, including the React Flow canvas, the live training visualisations, the admin operations dashboard, the i18n bundles for English and French, the high-contrast theme, and the impersonation banner.
- **14 docker-compose services** (5 application services, 3 Celery workers, 4 backing data stores, Ollama, ollama-pull, MailHog).
- **14 tabular ML algorithms** covering classification, regression, clustering, and time-series forecasting.
- **A fully-local RAG pipeline** built on sentence-transformers, pgvector, and Ollama serving Llama 3.2 3B Q4_K_M.
- **3 deep-learning architectures** (LeNet, custom ResNet-9, MobileNet-V3-Small with optional ImageNet transfer learning), guarded by a static VRAM estimator that refuses oversize requests before the GPU sees them.
- **A GitHub Actions CI pipeline** with parallel lint and test jobs across all five Python services and the TypeScript frontend.
- **An Alembic migration history** of nine revisions across two services, idempotent and replayable.

Validation results

The platform was validated end-to-end on two representative workloads. On a 250,000-row salary dataset with a Random Forest regressor, the system reached an R^2 of 0.9785 in 4 min 12 s on the demo machine — roughly 97.85% of the variance in the target explained by the features, on synthetic data where that level of fit is achievable but on real data would warrant a closer look at residual structure. On the synthetic 30-image, three-class dataset shipped as `seed_demo_image_dataset.py`, `tiny_resnet` at 64px batch 32 over five epochs reached terminal validation accuracy of 100% in roughly 30 seconds on a 1660 Super. The dataset is intentionally simple; the number proves the pipeline rather than the difficulty of the task.

What worked, what we would change

What worked. The single design decision that paid off most was the gateway-only header-injection rule from Sprint 1. Every downstream service can trust `X-User-Id` without inspection, which removed an entire category of cross-service auth bugs a more permissive setup would have invited. The shared-volume convention came second — it saved us from writing a streaming-upload protocol and generalised cleanly to RAG documents, image zips, and trained-model artefacts. The Celery-per-service split came third — a long XGBoost run cannot block an authentication request, and a 20-epoch DL run cannot starve the fast tabular queue.

What we would change. Flask in favour of FastAPI: the absence of typed routes cost time across the project's life, especially in `ml-training-service` where the request bodies are large and nested. Three algorithm registries instead of one: the tabular registry, the architecture registry, and the algorithm-display catalogue encode much of the same information in slightly different shapes, and a single config-driven catalogue would have been easier to maintain. A formal schema for socket events: the `/training` namespace grew organically and is mildly inconsistent (some events use `step`, some use `epoch`, some use both). A discriminated-union schema (Pydantic or a TypeScript literal type) would have caught the inconsistencies before they shipped.

Limitations and future work

The platform is single-host by construction — the shared-volume convention assumes a single filesystem. Scaling out the workers would need either a network filesystem or an S3-compatible object store. Per-user quota overrides require a JWT refresh to take effect because the tier rides in the JWT claim. The DL VRAM guard is calibrated against the demo machine; a truly portable estimator would instrument a one-time calibration run on first boot.

Future work, in approximate order of effort: ONNX export for DL versions (the `dl-training` service already saves a clean `state_dict` — this is a one-file change away); object detection via a pre-built YOLOv8 model for inference only; per-user quota override invalidation via a Redis-backed override cache the gateway consults on each request; and a typed schema for the `LiveTrainingChart` socket events, retrofitted as a discriminated union across the frontend and the worker.

Closing remark

The takeaway from this project is a specific belief: the common framing that “cloud AI is the only way to scale ML” is not exactly wrong, but it understates how much can be done locally. A laptop with 6 GB of VRAM, a competent CPU, and 16 GB of RAM is enough to host an AutoML platform, a small generative-AI workspace, and a deep-learning training loop — all at once, and without any of the data ever leaving the machine. For the regulated audiences this project was framed against, that is not a marginal improvement. It is the difference between machine learning being usable at all and not.

Bibliography

- [1] F. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [2] T. Chen and C. Guestrin, *XGBoost: A Scalable Tree Boosting System*, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.
- [3] S. M. Lundberg and S.-I. Lee, *A Unified Approach to Interpreting Model Predictions*, Advances in Neural Information Processing Systems 30 (NeurIPS 2017).
- [4] S. J. Taylor and B. Letham, *Forecasting at Scale*, The American Statistician, 2018.
- [5] W. Wang et al., *MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers*, NeurIPS 2020.
- [6] P. Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020.
- [7] Meta AI, *Llama 3.2: Open foundation and instruction models*, <https://ai.meta.com/llama/>, 2024.
- [8] A. Howard et al., *Searching for MobileNetV3*, Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2019.
- [9] K. He, X. Zhang, S. Ren, and J. Sun, *Deep Residual Learning for Image Recognition*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [10] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.
- [11] A. Paszke et al., *PyTorch: An Imperative Style, High-Performance Deep Learning Library*, NeurIPS 2019.
- [12] Ollama, *Get up and running with large language models locally*, <https://ollama.com/>, 2024.
- [13] A. Kane, *pgvector: Open-source vector similarity search for Postgres*, <https://github.com/pgvector/pgvector>, 2024.
- [14] xyflow GmbH, *React Flow Documentation*, <https://reactflow.dev/>, 2024.
- [15] Celery Project, *Celery: Distributed Task Queue*, <https://docs.celeryq.dev/>, 2024.